LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 5. Conditional Statements

Lecture: 36. Section Introduction

Section 1: Lecture Summary

This lecture introduces **conditional statements** in Python, which are essential for making decisions in programs by executing code based on conditions. It builds on previously covered topics like data types and arithmetic operators, emphasizing that conditional logic forms one of the basic pillars of programming along with data types and operators. The lecture covers writing **if-else** statements, using **relational and logical operators** with various data types, and constructing **nested conditional statements**. Students are encouraged to practice these concepts thoroughly through challenges and exercises using any Python environment or IDE, including pen and paper for conceptual clarity.

Section 2: Key Concepts and Explanations

- **Conditional Statements in Python**

  - Used to execute code blocks depending on conditions.

  - Basic forms include `if`, `if-else`, and `elif`.

- **If Statement**

  - Syntax:

code block if condition is True

```
    if condition:
```

  - Executes the code block only when the condition evaluates to True.

- **If-Else Statement**

- Syntax:

code block if condition is True

code block if condition is False

```
    if condition:
    else:
```

- Adds an alternative block of code to be executed when the condition is False.

- **Elif (Else If)**

- Allows checking multiple conditions sequentially.

- Syntax:

code block

code block

code block

```
    if condition1:
    elif condition2:
    else:
```

- **Relational Operators**

- Used to compare values.

- Common operators:

  - `==` equal to

  - `!=` not equal to

  - `<` less than

  - `>` greater than

  - `<=` less than or equal to

  - `>=` greater than or equal to

- **Logical Operators**

- Combine multiple boolean expressions.

- Operators:

  - `and` — returns True if both conditions are True.

  - `or` — returns True if at least one condition is True.

  - `not` — negates a boolean value.

- **Nested Conditionals**

  - Conditionals inside other conditionals to handle complex decision-making.

  - Indentation indicates the block of code related to each condition.

- **Conceptual Importance**

  - Although syntax and basic usage are simple, effective use of conditionals requires deep understanding for writing correct and efficient code.

- **Practice Recommendations**

  - Use any Python IDE or notebooks (like Jupyter).

  - Solve given challenges to reinforce learning.

  - Practice writing by hand if helpful.

Section 3: Example Code and Use Cases

Basic if statement example

```
x = 10
if x > 5:
    print("x is greater than 5")
```

Explanation: Prints a message only if x is greater than 5.

if-else statement example

```
x = 4
if x % 2 == 0:
    print("x is even")
else:
    print("x is odd")
```

Explanation: Checks if x is even or odd and prints accordingly.

if-elif-else statement example

```
score = 75
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: F")
```

Explanation: Assigns grades based on score ranges using multiple conditions.

Nested conditional example

```
x = 10
if x > 0:
    if x < 20:
        print("x is positive and less than 20")
```

Explanation: Checks two conditions nested inside one another.

Using logical operators

```
age = 25
if age >= 18 and age < 30:
    print("You are a young adult")
```

Explanation: Combines conditions with `and` to check if age is between 18 and 29.

Section 4: Key Takeaways

- Conditional statements (`if`, `if-else`, `elif`) control program flow based on boolean expressions.

- Relational operators compare values; logical operators combine conditions.

- Nested conditionals allow for complex decision trees.

- Mastery of these basics is crucial for efficient coding in any programming language.

- Practice using various conditions, nesting, and logical operators to build confidence.

- Challenges and hands-on coding (in any editor or on paper) help reinforce understanding.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.